SW Architecture

For

Smart Learn

Version <1.0.0>

Prepared by

Group Name: Team 2

Ricardo Nieto Campos	**217553941**	**ricardo.nieto5394@alumnos.udg.mx**
Ximena Montserrat González Sandoval	**220418567**	**ximena.gonzalez1856@alumnos.udg.mx**
Brenda Sarai Ramirez Aguilar	**224003663**	**brenda.ramirez0366@alumnos.udg.mx**
David Isaac Martinez Moreno	**224004236**	**david.martinez0423@alumnos.udg.mx**
Julia Vanessa Cortes Quezada	**221090669**	**julia.cortes9166@alumnos.udg.mx**

Instructor: **RUBEN ADRIAN GIL RIVERA**

Course: **Ingenieria de Software**

Lab Section: ***D13***

Date: **25 Marzo del 2026**

Table of Contents

1 PURPOSE 3

2 DEFINITIONS AND ABBREVIATIONS 3

3 REALIZATION CONSTRAINTS AND TARGETS 4

4 SW FUNCTIONAL ARCHITECTURE 5
4.1 Table of functions 6
4.2 Table of functional Interfaces 6
4.3 Functional Interface 7
4.4 Functional Interaction 7

5 SW PHYSICAL ARCHITECTURE 8
5.1 Physical Decomposition 8
5.2 Table of SW components. 8
5.3 Table of Physical Interfaces 9
5.4 Physical Interfaces 10

6 FUNCTION TO PHYSICAL ALLOCATION 11

7 SW REQUIREMENTS ALLOCATION 11

8 SW INTEGRATION PLAN 12

1 Purpose

El propósito de este documento es especificar los requerimientos de software para la versión 1.0 de la plataforma web Smart Learn. Este SWA abarca la totalidad del sistema central, incluyendo la capa de presentación (interfaz de usuario), la capa de lógica y la capa de persistencia de datos. Este documento servirá como la guía principal para el equipo de desarrollo durante la fase de codificación y como el criterio de evaluación principal para los profesores y revisores del proyecto.

2 Definitions and abbreviations

Abbreviations.

- **las operaciones básicas de base de datos:** Crear (Create), Leer (Read), Actualizar (Update) y Eliminar (Delete).
- **JSON (JavaScript Object Notation):** Formato ligero de intercambio de datos utilizado para la comunicación en el sistema.
- **JWT (JSON Web Token):** Estándar abierto para la transmisión segura de información entre partes, utilizado para la autenticación de usuarios.
- **SPA (Single Page Application):** Aplicación web que carga una sola página HTML y actualiza su contenido dinámicamente.
- **SRS (Spaced Repetition System):** Sistema de Repetición Espaciada, el algoritmo central de aprendizaje del proyecto.
- **COMET:** Se utiliza para estructurar la comunicación entre los componentes del backend y la gestión de procesos asíncronos en el stack.
- **UML:** Es el lenguaje estándar de la industria para visualizar, especificar, construir y documentar los artefactos de un sistema de software. Se emplea para crear los diagramas de casos de uso, de clases y de secuencia que describen el comportamiento dinámico de la plataforma.
- **API:** Application Programming Interface, Interfaz que permite la comunicación entre el Frontend y el Backend.
- **REST:** Representational State Transfer, Estilo de arquitectura usado para diseñar los endpoints del backend.
- **SM-2:** SuperMemo 2, Algoritmo de Repetición Espaciada que calcula los intervalos de repaso de las flashcards.
- **MVP:** Minimum Viable Product, Versión funcional mínima del sistema con las funciones esenciales.

- **ERD:** Entity-Relationship Diagram, Diagrama que representa las entidades de la base de datos y sus relaciones.

- **PERN:** PostgreSQL, Express, React, Node.js, Stack tecnológico completo del proyecto.

- **HTTP/HTTPS:** HyperText Transfer Protocol / Secure, Protocolo de comunicación entre cliente y servidor.

- **SQL:** Structured Query Language, Lenguaje utilizado para consultas a la base de datos PostgreSQL.

- **SDD:** Software Design Document, Documento de diseño de software (referenciado desde el SDD hacia el SWA).

References

1. IEEE Std 830-1998, Práctica Recomendada por IEEE para Especificaciones de Requerimientos de Software
2. Documentación oficial de React.js (UI)
3. Documentación oficial de Node.js y Express (server)

Documentación técnica de Supabse y PostgreSQL (DataBase & Autentication)

3 Realization constraints and targets

- Operación correcta de un Sistema centralizado que administre usuarios, categorías de estudio y tarjetas de memoria (flashcards).
- Un sistema capaz de ejecutar la lógica matemática de un algoritmo de repetición espaciada (SM-2) para optimizar los tiempos de estudio de los alumnos de manera personalizada.
- Un Sistema basado en una arquitectura Cliente-Servidor (Frontend en React.js, Backend en Node.js/Express.js).
- Un Sistema cuya persistencia de datos esté restringida y asegurada en una base de datos PostgreSQL alojada en la infraestructura de Supabase
- Un Sistema que opere bajo el protocolo Seguro HTTP/HTTPS, utilizando exclusivamente el formato JSON para la transferencia de datos.
- **Restricción de Almacenamiento:** El Sistema cuenta con una restricción estricta de memoria limitado por el tamaño máximo por recurso físico almacenado (PDFs e imágenes) a **10MB por archivo** para asegurar el rendimiento de la plataforma.
- **Dependencias y Suposiciones:** El Sistema asume una conectividad a internet estable por parte del cliente y tiene una dependencia crítica de disponibilidad con la infraestructura en la nube de Supabase (PostgreSQL y Auth).

4 SW Functional Architecture

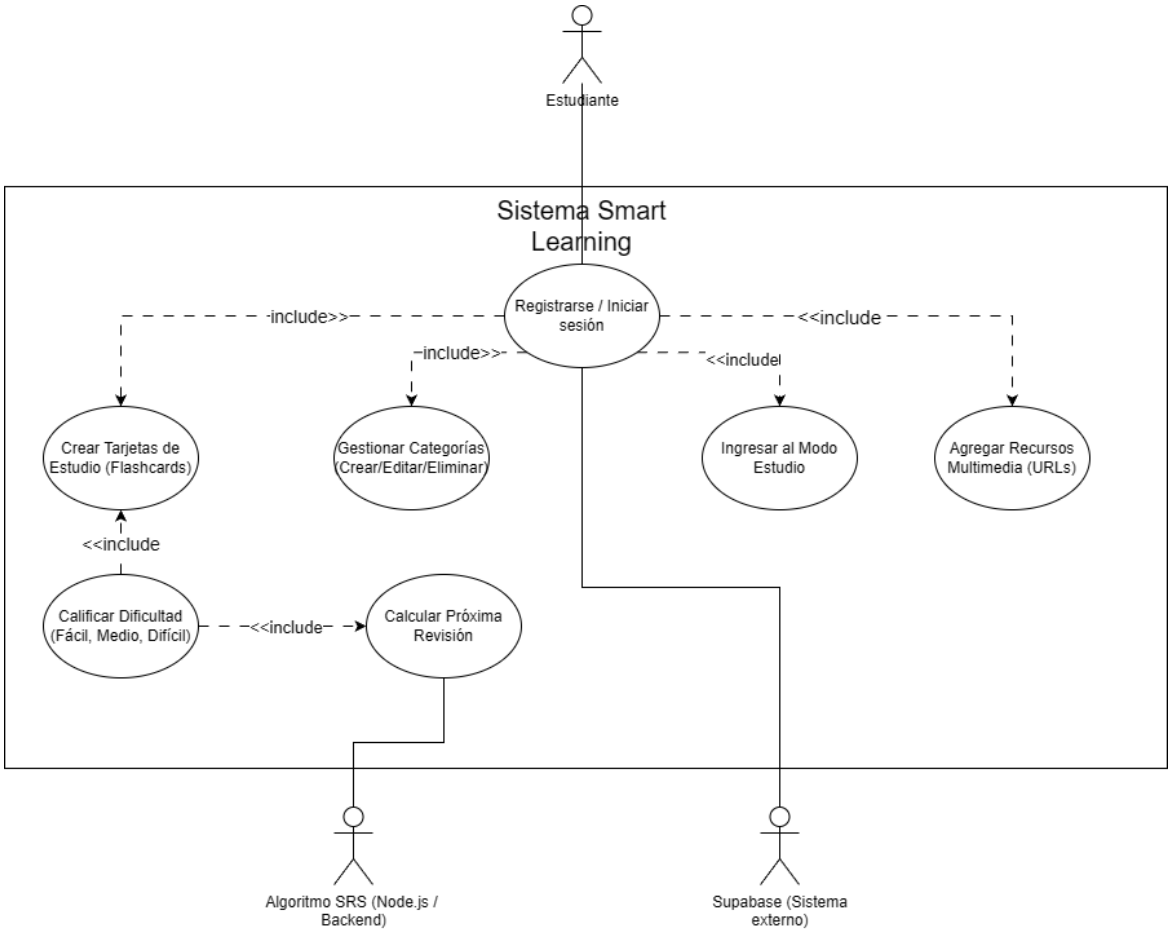

Figura 1.0 Diagrama de Casos de Uso

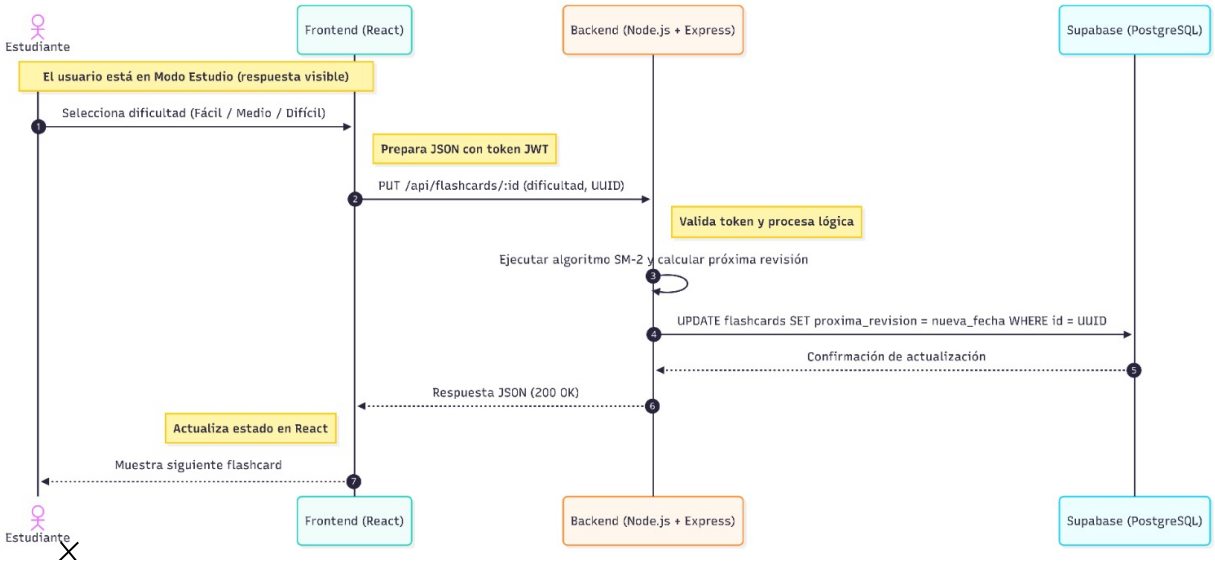

Figura 2.0 Diagrama de Casos de Secuecia de Dificultad

4.1 Table of functions

Function Name	Description
F-01: Autenticación	Gestiona el registro de nuevos usuarios, el inicio de sesión y la validación de identidad mediante tokens.
F-02: Gestión de Estructura	Permite al usuario organizar su aprendizaje mediante la creación, edición y eliminación de categorías de estudio.
F-03: Gestión de Recursos	Almacena y vincula enlaces externos (URLs) de materiales y PDF's.
F-04: Generación de Flashcards	Transforma la información de los recursos en tarjetas interactivas de pregunta y respuesta.
F-05: Procesamiento SRS	Ejecuta el algoritmo de repetición espaciada para determinar la prioridad y fecha del próximo repaso.
F-06: Filtrado de Dashboard	Selecciona y muestra únicamente las tarjetas que requieren atención inmediata según la fecha actual.

4.2 Table of functional Interfaces

Signal/Event	Description
HTTP Request (JSON)	Evento disparado por el cliente (Frontend) para solicitar o enviar datos al servidor.
Auth Token (JWT)	Señal de seguridad que acompaña a cada petición para validar permisos de acceso.
Database Query (SQL)	Intercambio de información entre el backend y la base de datos PostgreSQL.
Dificultad_Input	Valor numérico (1-3) enviado por el usuario que activa el cálculo del algoritmo SRS.
Respuesta_Algoritmo	Nueva fecha de revisión calculada enviada desde el servidor para actualizar el estado de la tarjeta.

4.3 Functional Interface

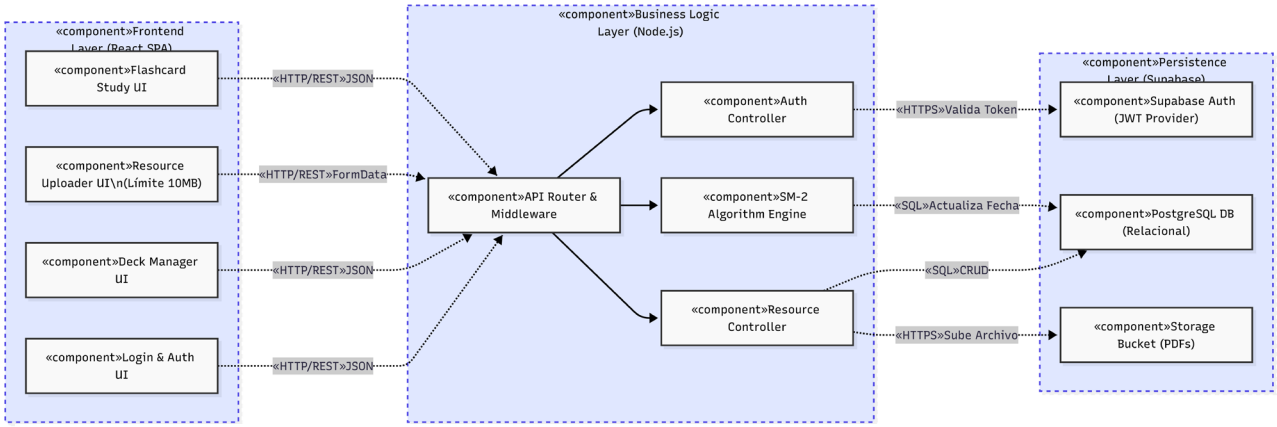

Figura 3.0 Diagrama de componente (Component Diagram)

4.4 Functional Interaction

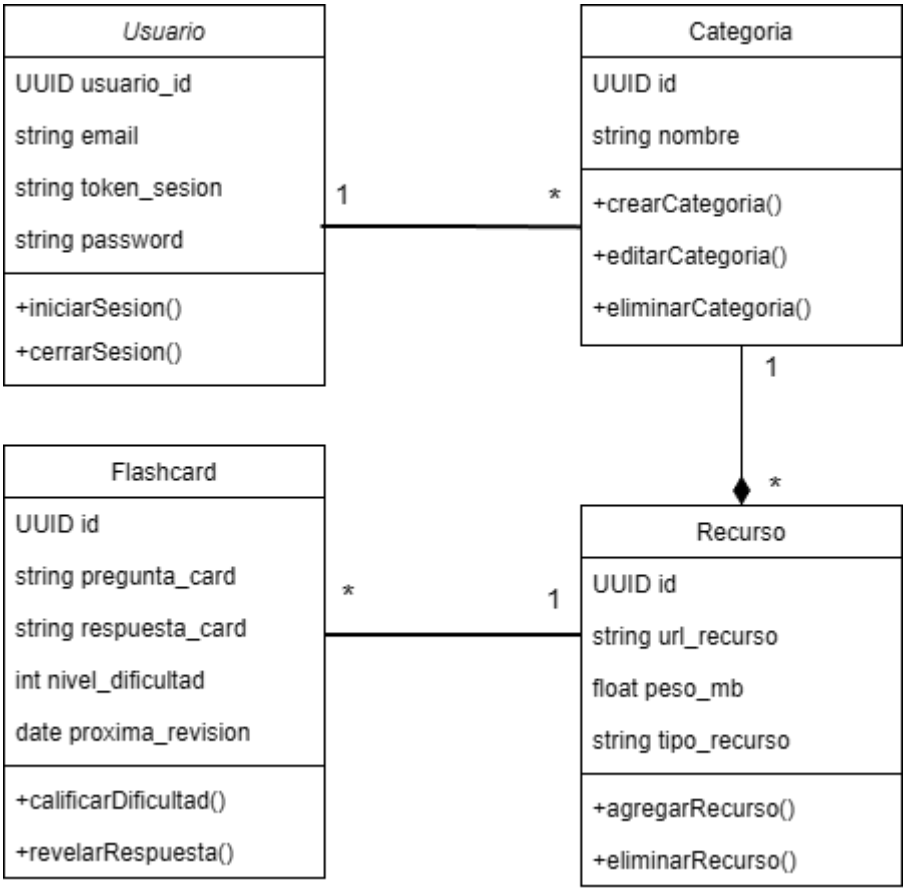

Figura 4.0 Diagrama de clases

5 SW Physical Architecture

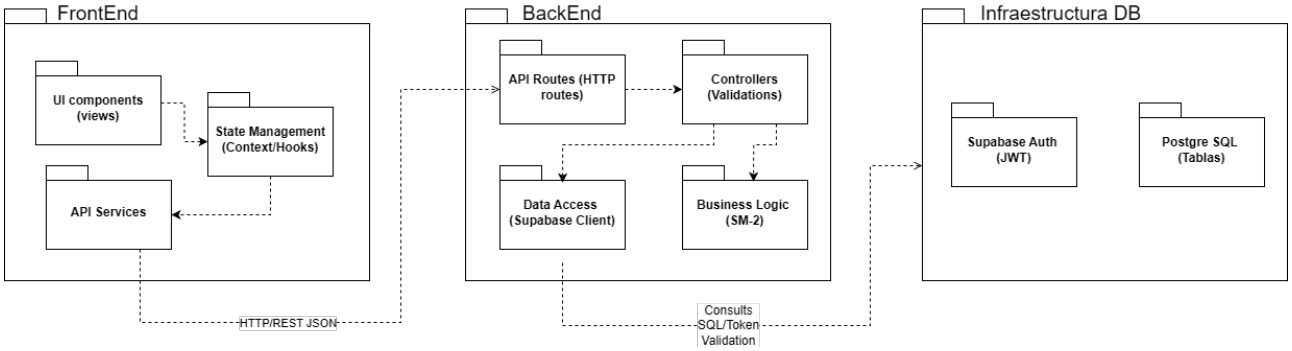

Figura 5.0 Diagrama de paquetes (Diagram Packege)

5.1 Physical Decomposition

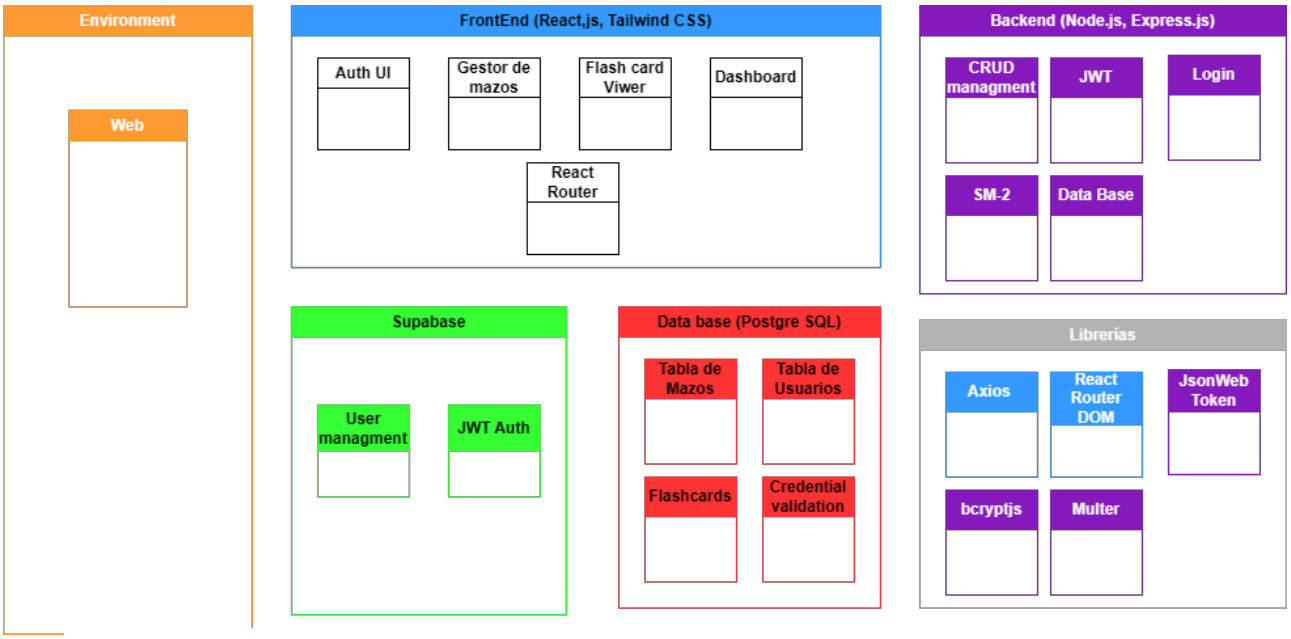

Figura 5.1 Diagrama de Descomposición Física (Diagram physical Decomposition)

5.2 Table of SW components

SWC Name	Description	Estimated Size (KB)
SWC_01: React Frontend	Aplicación de una sola página (SPA) que gestiona la lógica de vista y el estado local del usuario.	1
SWC_02: Express REST API	Servidor de aplicaciones que procesa las peticiones, ejecuta el algoritmo SRS y valida la seguridad.	1

SWC_03: Supabase DB	Instancia de base de datos relacional PostgreSQL para la persistencia de datos.	1
SWC_04: Supabase Auth	Módulo externo encargado de la gestión de identidades y generación de tokens JWT.	2

5.3 Table of Physical Interfaces

Operation	Description
Cloud Hosting (Server)	Instancia con soporte para Node.js.
Client CPU/RAM	Capacidad estándar para ejecución de JavaScript en navegadores modernos.
Storage	Espacio en disco en la nube para la base de datos PostgreSQL.

5.4 Physical Interfaces & Interaction

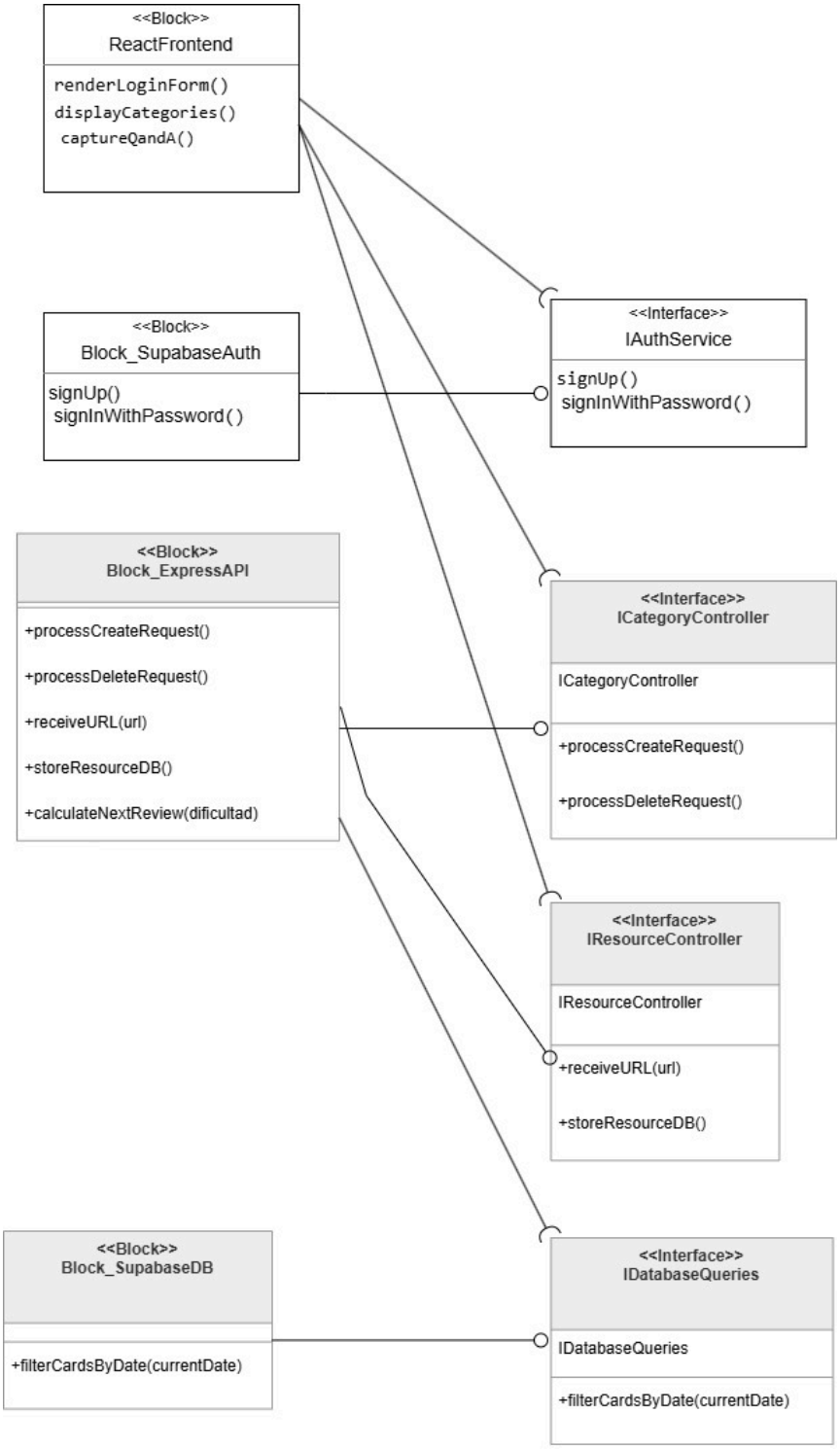

Figura 6.0 Diagrama de Descomposición Física (Diagram physical Decomposition)

6 Function to Physical Allocation

Function	SW component	Clarification
F-01: Autenticación	SWC_04 (Supabase)	El componente externo gestiona el hash de contraseñas y la emisión de tokens. El Frontend (SWC_01) provee el formulario.
F-02: Gestión de Estructura	SWC_01 (React) / SWC_02 (Node)	El Frontend gestiona el estado de la vista de categorías, mientras el Backend procesa las peticiones de creación/borrado.
F-03: Gestión de Recursos	SWC_02 (Express API)	El backend recibe las URLs y las envía a la base de datos (SWC_03) para su almacenamiento.
F-04: Generación de Flashcards	SWC_01 (React)	El frontend captura los campos "Pregunta" y "Respuesta" mediante formularios antes de enviarlos al servidor.
F-05: Procesamiento SRS	SWC_02 (Express API)	El algoritmo de repetición espaciada reside exclusivamente en el backend para garantizar la integridad de los cálculos de fecha.
F-06: Filtrado de Dashboard	SWC_03 (Supabase)	La lógica de filtrado se ejecuta mediante consultas SQL que seleccionan solo los registros cuya fecha de revisión sea igual o menor a la actual.

7 SW Requirements Allocation

Elements located in SW Project repository path:

8 SW Integration Plan

Integration Step	Description	SW components	Comments
1.- Persistencia y Datos	Configuración del esquema de tablas en PostgreSQL.	**SWC_03 (Supabase DB)**	Paso crítico para permitir el almacenamiento de usuarios.
2.- Core del Servidor	Implementación de la API básica y conexión con la base de datos.	**SWC_02 (Express API)**	Se verifica la conectividad mediante herramientas como Postman.
3.- Módulo de Acceso	Integración del servicio de autenticación con las rutas protegidas del backend.	**SWC_04 (Supabase Auth)**	Asegura que solo usuarios válidos usen la API.
4.- Interfaz y Navegación	Construcción de la SPA y configuración de rutas (Dashboard/Login).	**SWC_01 (React Frontend)**	Utiliza datos estáticos (mockup) antes de la conexión final.
5.- Integración de Funciones	Conexión del algoritmo SRS del backend con el Modo Estudio del frontend.	**Todos los componentes**	Es el punto donde el sistema se vuelve funcional según los requerimientos.